

ПИ-ТУ-ХОН И ПАМЯТЬ

как оно там вообще работает

про что разговор

- куда питон складывает объекты
- зачем у него свой аллокатор
- как он понимает, когда чистить
- зачем нужен GIL
- что получаем в итоге

где что лежит

два региона памяти

STACK

- локальные переменные
- живут пока функция жива

← вызовы функций
(точнее – указатели)

HEAP

- всё, что переживает конкретный вызов

← все объекты
int, str, list, твои классы

В ПИТОНЕ ВСЁ НА КУЧЕ

в С можно так: `int x = 5` — на стеке.

в ПИТОНЕ — **нельзя**. вообще.

почему: тип переменной может меняться, размер не предсказуем,
объект может уехать в коллекцию или замыкание.

→ всё валится в кучу, на стеке только **фреймы** (записи «я сейчас в функции X»).

руmalloc

свой аллокатор

зачем не просто malloc

питон создаёт **миллионы** мелких объектов в секунду.

обычный `malloc()` для такого медленный — он универсальный.

идея: взять у ОС один большой кусок, потом самим резать и раздавать.

это и есть **pymalloc**.

иерархия аллокации в питоне

Layer 3: object-specific (free lists)
отдельные корзинки для int, list, dict, tuple, ...

↓ если корзина пуста

Layer 2: pymalloc – блоки до 512 байт
свой аллокатор, без syscall

↓ если объект > 512 байт

Layer 1: PyMem_RawMalloc – обёртка над malloc

↓

Layer 0: OS – malloc / mmap / brk

каждый слой ловит свой класс запросов и не дёргает нижний без нужды.

внутри rtmalloc

arena (256 KB) — один кусок от ОС

pool (4 KB) — одна страница

block 24 byt	block 24 byt	block 24 byt	...
-----------------	-----------------	-----------------	-----

все блоки в pool — одного размера

pool, pool, pool... (64 штуки на arena)

аллокация — **O(1)** без системных вызовов.

free lists

ещё уровень оптимизации: для часто создаваемых типов
питон держит «корзинки на возврат».

создал int	→	взяли из корзинки int-ов
удалил int	→	кинули обратно
создал tuple	→	из корзинки tuple
удалил tuple	→	обратно

корзинки есть для int, float, list, dict, tuple, frame, ...

мгновенно, мимо pymalloc вообще.

КОГДА ЧИСТИТЬ

главный вопрос

у нас есть память, мы умеем выделять.

когда освобождать?

два подхода:

- **считать ссылки** → счётчик = 0 → освобождаем
- **периодически сканировать** → что недостижимо → мусор

ПИТОН взял **первый** + второй как страховка.

refcount как он есть

```
a = [1, 2, 3]      # refcount = 1
b = a              # refcount = 2
del b              # refcount = 1
del a              # refcount = 0 → ОСВОБОДИТЬ
```

у каждого объекта счётчик.

ссылка появилась — `++`. ссылка пропала — `--`.

плюс: объект исчезает сразу, как только не нужен.

предсказуемо, никаких «когда-нибудь потом».

проблема: циклы

```
a = []  
b = []  
a.append(b)    # a → b  
b.append(a)    # b → a  
  
del a, b        # внешних ссылок нет  
                # но refcount(a) = 1 (b на него смотрит)  
                # и refcount(b) = 1 (a на него смотрит)  
                # → утечка
```

refcount **не может** это разрулить в принципе.

решение: GC поверх

второй механизм — **сборщик циклов**.

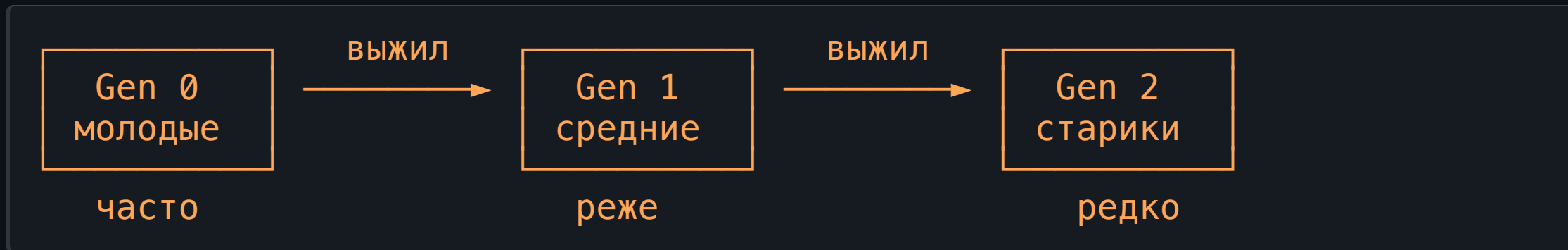
GC периодически:

1. смотрит на контейнеры (`list`, `dict`, классы)
2. считает «внутренние» ссылки в группе
3. находит замкнутые группы без выхода наружу
4. разрывает их

`int`, `str`, `float` он **не трогает** — у них циклов быть не может.

поколения

эмпирика: молодые объекты обычно умирают быстро.



молодых сканируем часто, старых — почти не трогаем.

GIL

зачем он

refcount **не атомарный**. в машинном коде это:

1. прочитать счётчик
2. прибавить 1
3. записать обратно

два потока одновременно:

```
T1: прочитал 5  
T2: прочитал 5  
T1: записал 6  
T2: записал 6      ← должно быть 7!
```

→ потеряли инкремент → объект освободится раньше → segfault

два решения

1) атомарный refcount — каждое изменение через специальную CPU-инструкцию.

работает, но **в 5–10 раз медленнее**.

2) глобальный замок — один поток за раз внутри питона.

single-threaded быстрый, multi-threaded — последовательный.

питон выбрал второе. это и есть **GIL**.

| решение принято в **1992 году**, когда многоядерных CPU не было.

ЧТО ЭТО ЗНАЧИТ НА ПРАКТИКЕ

CPU-задача (числа крутить):	потоки не помогают ❌
I/O-задача (сеть, диск):	потоки помогают ✅ (GIL отпускается на блокировке)

если CPU-параллелизм всё-таки нужен:

- `multiprocessing` — отдельные процессы
- `numru` и другие, отпускающие GIL внутри C-кода
- Python 3.13+ без GIL (экспериментально)

собираем картину

каждый слой решает свою проблему

объекты на куче	← питон динамический, размер/тип не предсказать
pyalloc	← malloc слишком медленный для миллиона мелочи
free lists	← даже pyalloc медленный для самых горячих типов
refcount	← нужно детерминированное удаление
GC поверх	← refcount не справляется с циклами
GIL	← refcount ломается в потоках без синхронизации

одно предложение про всё

всё в питоне — это **объекты на куче**, разложенные **своим хитрым аллокатором** (потому что malloc медленный), с **счётчиком ссылок** (для предсказуемой очистки), с **доп. сборщиком циклов** (которые refcount не вытащит), под **глобальным замком** (иначе счётчики ломаются), с кучей **кешей** (иначе всё это всё равно тормозит).

каждая штука существует **строго потому**, что без неё что-то ломается или становится непрактично медленно.

цена и выгода

✓ что получаем

- не думаешь о памяти
- безопасно (нет UAF)
- огромная экосистема
- быстрое прототипирование

✗ что платим

- объект 16+ байт overhead
- двойная индирекция
- GIL давит CPU-параллелизм
- 100–1000× медленнее C на примитивах

для CPU-числодробления → numpy / numba / multiprocessing.

для всего остального → питон отличный.